Cloud Web Server Security Report

Name: [Student Name]
Date: [Date]
Role: Junior DevOps Engineer

Understanding Customer Challenges and Requirements

Client Challenges

- **Current Setup and Issues**: Document the client's current webserver setup (a local webserver using Docker) and describe its limitations.

  The client uses a standalone local web server hosted in a Docker container.

  - *Instructions*: Explain why this local-only setup presents challenges for scalability, compliance, or disaster recovery.

    The local-only setup creates a single point of failure. The host machine fails, and the entire application goes offline. Furthermore, the container has a fixed resource limit, meaning it cannot handle sudden traffic spikes.

Audit Results

- **Baseline Security Posture**: Record the **hardening index score** from the Lynis audit and summarize the vulnerabilities identified.

  virtual box_udesktop22 1_09_05_2026_02_32_30.jpg

  - *Instructions*: Connect the vulnerabilities to the client's business challenges. For example, how does missing HTTPS impact compliance or disaster recovery?

    The absence of a firewall means a client can not detect or investigate a breach. Insecure configuration directly threatens compliance standards that require strict data protection and audit trails.

Evaluating Cloud Service Models

Service Model Comparison

- **Analysis of Models**: Compare the client's responsibilities under IaaS, PaaS, and SaaS models.

  IaaS (Infrastructure as a Service): the client remains responsible for the OS, firewall, and web server hardware.

  PaaS (Platform as a service): The provider manages the OS; the client only secure the application and data.

  SaaS (Software as a service): the provider handles nearly all security responsibilities.

  - *Instructions*: Identify which vulnerabilities (e.g., missing file permissions or insecure configurations) would remain the client's responsibility under each model.

    IaaS

Recommended Model

- **Chosen Service Model**: Recommend a cloud service model (e.g., IaaS, PaaS, or SaaS) based on the client's goals.

  IaaS

  - *Instructions*: Justify your recommendation by explaining how this model addresses the client's scalability, compliance, or security concerns.

    IaaS because it allows the client to have full control over specific security configuration, such as custom CIS Benchmark and Lynis remediation requirements.

Exploring Deployment Options

Deployment Choices

- **Analysis of Deployment Options**: Compare public, private, and hybrid cloud options.

  Public Cloud: Offers massive scalability but requires the strictest security measures (Hardening) because the infrastructure is shared.

  Private Cloud: Offers high security but is expensive and lacks the "on-demand" scaling of public options.

Hybrid Cloud: Combines both, keeping sensitive data local while using the public cloud for web traffic.

- *Instructions*: Explain how each option impacts scalability, security, and disaster recovery. For example, discuss why a public cloud might require stricter security measures.

Recommended Deployment Option

- **Chosen Deployment Option**: Recommend a deployment option for the client (e.g., public cloud for scalability, hybrid for flexibility).

  Public Cloud

  - *Instructions*: Justify your choice based on the client's challenges and the vulnerabilities identified in the audit.

    A Public Cloud deployment is recommended to solve the client's scalability issues. The security risks are mitigated by applying the "Golden Image" hardening process developed during the lab.

Developing a Cloud Solution Recommendation

Hardening Actions

- **Actions Taken**: Document the steps you took to remediate vulnerabilities on the local web server.
  - *Instructions*: For each vulnerability, provide the following:
    - **The Issue**: Describe the vulnerability (e.g., missing HTTPS, weak file permissions).

      Weak permissions

      Information leakage

      Insecure firewall

    - **The Action**: Document the exact commands or configurations used to address the issue.

      sudo cServerTokens Prod in /etc/apache2/conf-enabled/security.confhmod 750 /var/www/html

      sudo ufw allow 80/tcp and sudo ufw enable

        - **The Result**: Explain how the remediation improves security.

          Prevents unauthorized users from modifying web files.

          Hides Apache version info from attackers.

          Closes all ports except those explicitly required for web traffic.

Cloud Solution Recommendation

- **Recommendation**: Propose a cloud solution that integrates the lessons learned from the hardening process.

  Hardened IaaS solution using an Auto-Scaling Group.

  - *Instructions*: Justify your recommendation by explaining how the chosen cloud solution (e.g., public cloud with additional hardening) addresses the client's scalability, security, and disaster recovery needs.

    By creating a pre-hardened Virtual Machine image (based on your Ubuntu 22.04 remediations), the client can deploy identical, secure instances that scale automatically based on demand.

Disaster Recovery Planning

Proposed Recovery Plan

- **Backups**: Outline a plan for backing up the web server's configurations and data.

  Automated Snapshots of the Ubuntu volumes

  - *Instructions*: Explain how regular backups protect the client from data loss and improve recovery time.

    This ensures that even if a server is compromised, it can be restored to a clean state in minutes.

- **Failover Mechanisms**: Describe strategies for high availability (e.g., redundant web servers, load balancing).

  Use a Load Balancer across multiple availability zones

    - *Instructions*: Explain how these mechanisms ensure the web server remains available during a failure.

      If one zone fails, traffic is rerouted instantly to a healthy server.

- **Monitoring and Alerts**: Propose logging and monitoring tools to detect and respond to issues proactively.

  Prometheus or CloudWatch

    - *Instructions*: Explain how monitoring contributes to disaster recovery and reduces downtime.

      It alert the team when a hardening policy is violated or a service goes down.

Connection to Hardening Actions

- **How Hardening Supports Recovery**: Explain how the security remediations you applied (e.g., enabling HTTPS, configuring permissions) contribute to disaster recovery goals.
    - *Instructions*: Tie each remediation to a recovery objective (e.g., secure backups, faster recovery).

      Hardening a system is very important to develop a stable and predictable baseline to support a successful Disaster Recovery (DR) plan. The fewer points of attack you have, the fewer points of attack that are available when something goes wrong; the fewer points of attack the fewer points of attack to come back from, in case of failure.

        - ☐ Strict File & Directory Permissions: With strict permissions you can be assured of Backup Integrity. This will not allow unauthorized users to change Web files, so when you have to restore from a backup, you can be sure you aren't restoring "poisoned" data or hidden malware.
        - ☐ Information Masking (Server Tokens/Banners): Masking certain version numbers results in Faster Recovery. It decreases the chance that the successful exploitation will happen and in the cloud with a hardened "Golden Image", a clean and secure instance can be redeployed without manual re-configuration in minutes.
        - ☐ Firewall Configuration & Logging: As you've seen in the Lynis audit results, it is essential to check for rulesets and logging when investigating an Incident for Incident Isolation. If proper firewalling is used, then the threats will not move across the nodes reducing

the scope of the disaster and limit recovery efforts to a single node.

- ☐ Secure Failover is enabled by having HTTPS: Securing data in transit enabled. If, during a disaster, traffic is being shifted to a backup region, using HTTPS preconfigured ensures that the traffic continues to be encrypted and secure while in the backup region.

Reflection on the Cloud Process

Reflection on Each Phase

- **Understanding Challenges**:
  - *Instructions*: Reflect on how analyzing customer challenges helped you understand their needs and guided your actions.

    A local web-server in Docker was analyzed as the current implementation and this identified the critical need for high availability. I was conscious that although technical issues were important, the actual issues that the client was experiencing were scalability and the "single point of failure" risk which led me to think in a cloud first mindset.

- **Evaluating Service Models**:
  - *Instructions*: Reflect on what you learned about cloud service models and how they impact security responsibilities.

    IaaS, PaaS and SaaS helped to make the Shared Responsibility Model clear. I learned that for this particular client, they needed IaaS in order to have complete control over their OS-level hardening, and they needed to meet specific CIS Benchmarks.

- **Exploring Deployment Options**:
  - *Instructions*: Reflect on how considering deployment options helped you align your recommendations with customer goals.

    I took a look at public, private and hybrid clouds to meet technical security goals with business goals. I realized that in a public cloud, the most efficient way to scale up is to use the public cloud, but you have to do the most stringent hardening of the cloud to ensure security from the threat that comes from shared infrastructure.

- **Hardening and Recommendations**:
  - *Instructions*: Reflect on the impact of the hardening process and how it informed your recommendation for a scalable cloud solution.

The hands-on hardening process had the most impact. Scalability was proven to be the base of security as they moved from the initial Lynis audit to the remediated state. Therefore, my recommendation is: Just a Cloud solution will be truly "scalable" if it is based on a "Golden Image" which is hardened and repeatable.

- **Disaster Recovery**:
    - *Instructions*: Reflect on the importance of disaster recovery and how it connects to the hardening actions you applied.

      A look at DR reminded me of the fact that Security and Availability were two sides of the same coin. Hardening measures such as setting up permissions and firewalls would not only protect the system, but they would also be recovery measures that would keep the system secure and recoverable in the event of failure.

Overall Reflection

- Summarize your key takeaways from the lab.
- *Instructions*: Discuss how this process can be applied to real-world cloud security scenarios, and what you learned about balancing technical tasks with strategic decision-making.

  This lab proves that security is a part of DevOps and should be an integral component at a base level. I have learned how to make a liability a "Golden Image" using the cloud by taking a "local" server, auditing it with Lynis and then using CIS Benchmarks.

  Key Takeaway: Technical Tasks (such as hardening the CLI) and Strategic Decisions (such as selecting IaaS over PaaS to ensure compliance control) need to be managed when putting security in place in the real world.